

Schedulr — Smart Timetable Generator

Comprehensive Project Documentation

Problem Statement: PS1 — *"Develop an intelligent timetable generation system that creates conflict-free schedules while considering faculty availability, classrooms, laboratories, batches, electives, and scheduling constraints."*

Team Repository: <https://github.com/Vedant1703/smart-timetable-generator>

Table of Contents

1. [Executive Summary](#)
2. [Problem Analysis](#)
3. [Solution Overview](#)
4. [System Architecture](#)
5. [Technology Stack & Justification](#)
6. [Data Model](#)
7. [The Constraint Engine \(Core Innovation\)](#)
8. [Natural Language Rule Builder](#)
9. [The Solver Pipeline](#)
10. [Independent Conflict Verification](#)
11. [Role-Based Access Control](#)
12. [Core Workflows](#)
13. [Multi-Tenant Architecture & Security](#)
14. [API Design](#)
15. [Frontend Design](#)
16. [Testing Strategy & Results](#)
17. [Performance Characteristics](#)
18. [What Makes Schedulr Different](#)
19. [Future Roadmap](#)

1. Executive Summary

Schedulr is a multi-tenant SaaS scheduling engine that generates mathematically guaranteed conflict-free academic timetables. An institution describes its reality once — who can teach what, which rooms exist and what equipment they have, how cohorts split into batches and electives, what rules must never be broken — and the system returns a timetable that is:

- **Provably correct** — verified by an independent conflict-checker (separate code path from the solver)
- **Optimally balanced** — soft preferences like faculty load balancing and room utilization are optimized via weighted objectives
- **Human-approved** — every generated timetable must pass through a mandatory Review → Approve → Publish workflow before becoming visible

The actual differentiator is not "it draws a grid." It is that the grid is provably clash-free at a scale no human scheduler can hold in their head, and it's re-proved cheaply every time something changes.

2. Problem Analysis

The Real-World Scheduling Challenge

Academic timetable scheduling is an NP-hard combinatorial optimization problem. A medium-sized college with 40 cohorts, 100+ faculty, 50+ rooms, and 200+ course-cohort assignments faces a search space of billions of possible slot allocations. Manual scheduling — the method most institutions still use — suffers from:

Challenge	Impact
Faculty double-booking	A professor accidentally scheduled for two classes at the same time
Room conflicts	Two cohorts assigned to the same room simultaneously
Workload violations	Faculty exceeding their weekly/daily teaching caps
Elective clashes	A student's chosen electives overlapping with their core courses
Lab scheduling	Multi-period lab blocks split across non-consecutive slots or different rooms
Iterative conflicts	Fixing one clash creates new cascading clashes elsewhere
Scale	Impossible to mentally verify all constraints across 200+ assignments

Why Existing Tools Fall Short

Most scheduling tools use either:

- **Greedy heuristics** — fast but produce sub-optimal results with frequent constraint violations
- **Manual drag-and-drop** — human-driven, doesn't scale, and provides no formal correctness guarantee
- **Rule-of-thumb algorithms** — cannot express the full complexity of real institutional constraints

Schedulr uses **constraint programming** — the same mathematical formalism used in airline crew scheduling and industrial logistics — to model and solve the problem with formal guarantees.

3. Solution Overview

Schedulr is built around three core principles:

Principle 1: Mathematical Correctness Over Heuristic Approximation

Every generated timetable satisfies all 15 hard constraints by construction (enforced within the solver) and is independently re-verified by a separate conflict-checker module that uses a completely different code path.

Principle 2: Human Control Over Automation

- Natural language rules are parsed but **never auto-applied** — they require explicit confirmation
- Generated timetables are **never auto-published** — they require reviewer approval
- The confirmation text shown to users is **template-rendered from structured data**, never re-generated by the LLM

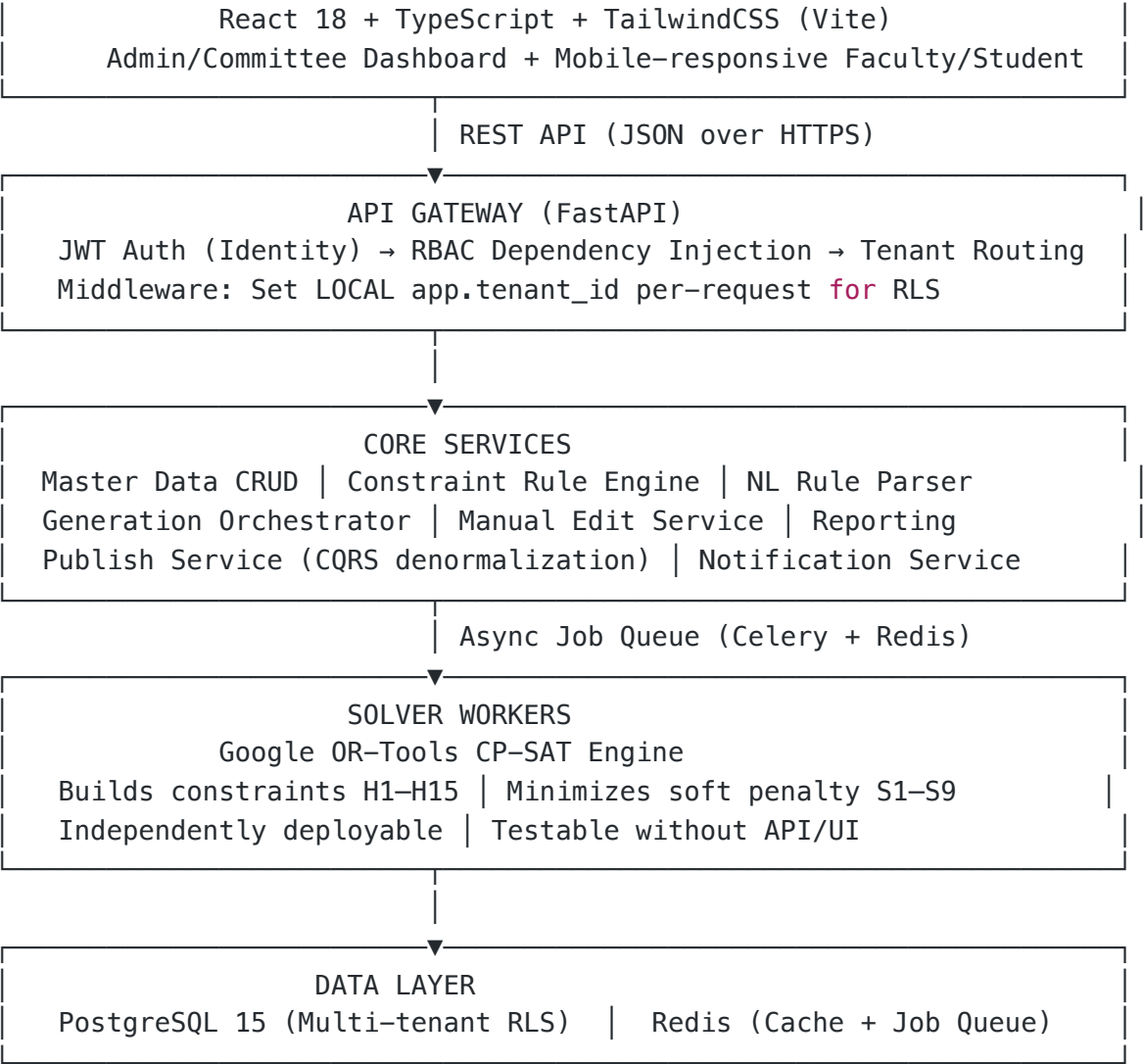
Principle 3: Tenant Isolation by Design

Data security is enforced at the database layer via Postgres Row-Level Security (RLS), not just at the application layer where a forgotten `WHERE` clause could leak data.

4. System Architecture

4.1 Five-Layer Architecture





4.2 Key Architectural Decisions

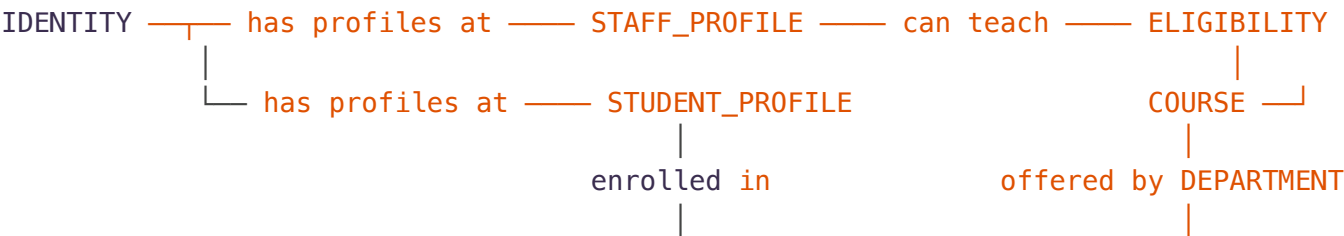
Decision	Rationale
Solver as independent module	Can be tested, scaled, and deployed independently of the API and UI (NFR-19)
CQRS-lite pattern	Generation writes raw assignment rows; Publish denormalizes into per-view read tables for fast queries
Warm-started incremental regeneration	CP-SAT AddHint() seeds re-solves from the previous solution, so small changes don't restart the full solve
Independent conflict checker	A completely separate code path re-validates H1-H15 after every solve — we don't just trust the solver
Database-layer tenant isolation	RLS policies on every table, not just app-layer WHERE tenant_id = ... filters

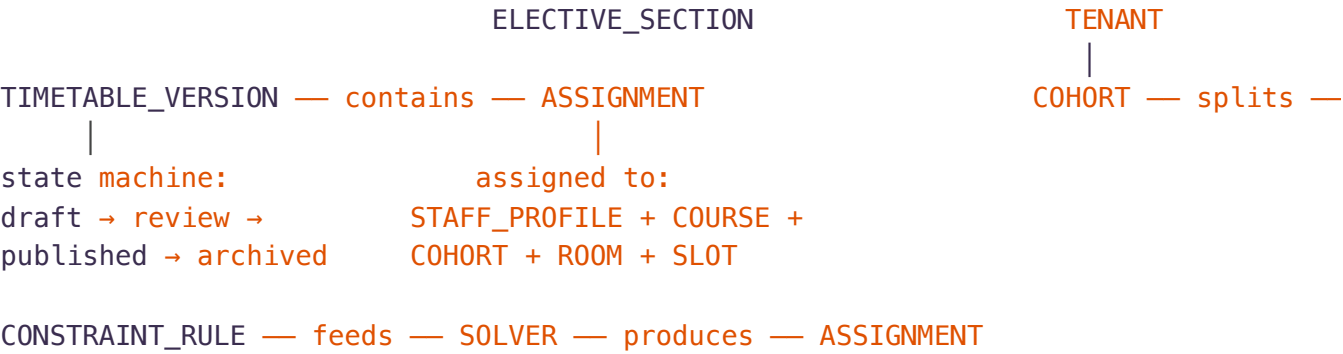
5. Technology Stack & Justification

Layer	Technology	Why This Choice
Frontend	React 18 + TypeScript + TailwindCSS (Vite)	Component model ideal for the timetable grid; Tailwind enables rapid UI iteration
API	Python 3.11+ (FastAPI) + SQLAlchemy (Async)	Same language as the solver eliminates cross-language serialization overhead for large constraint models
Solver	Google OR-Tools CP-SAT	Industry-grade constraint solver; supports solution hints for incremental regen; free and open-source
Rule Parsing	OpenAI API with constrained JSON output schema	Fastest path to NL rule understanding without custom model training
Database	PostgreSQL 15+	Join-heavy relational model; native Row-Level Security; JSONB for flexible fields
Job Queue	Celery + Redis	Async solver execution for large institutions; Redis also serves as cache
Auth	JWT + OAuth2 (Keycloak)	Supports one Identity spanning multiple tenants; standard OIDC flows
Containerization	Docker Compose	Single-command demo environment; production path to Kubernetes
Migrations	Alembic	Version-controlled schema evolution; supports RLS policy management

6. Data Model

6.1 Entity Relationship Overview





6.2 Core Tables

Table	Purpose	Key Columns
identity	Person-level login (tenant-independent)	id, email, auth_provider_ref, platform_role
tenant	Institution with isolated config	id, name, institution_type, timezone, enabled_modules
campus	Multi-campus support	id, tenant_id, name, timezone
department	Organizational unit	id, tenant_id, name
course	Academic offering	id, tenant_id, department_id, type (core/elective/lab), hours_per_week, block_size
staff_profile	Tenant-scoped faculty record	id, identity_id, tenant_id, workload_cap_week, workload_cap_day, roles
student_profile	Tenant-scoped student record	id, identity_id, tenant_id, cohort_id
cohort	Student group	id, tenant_id, name, type (fixed/elective_derived)
batch	Sub-group for labs	id, cohort_id, course_id, label
room	Physical space	id, tenant_id, campus_id, type, capacity, equipment_tags, accessible

Table	Purpose	Key Columns
eligibility	Faculty-course-cohort mapping	staff_profile_id, course_id, cohort_id (composite key)
constraint_rule	Scheduling rule	id, tenant_id, rule_type, scope, target_id, weight, source, status
period_template	Time slot definitions	id, tenant_id, weekday, period_index, start_time, end_time
staff_availability_block	Faculty unavailable/admin periods	id, staff_profile_id, weekday, period_index, block_type
timetable_version	Schedule state machine	id, tenant_id, state (draft/under_review/published/archived), version_no
assignment	Solver output (write model)	id, timetable_version_id, staff_profile_id, course_id, cohort_id, room_id, slot_start, slot_span
published_schedule	Denormalized read model (CQRS)	Optimized for per-faculty, per-student, per-room view queries
audit_log	Change tracking	id, tenant_id, actor, action, entity_type, before/after (JSONB)

6.3 The Two Most Critical Tables

```
-- The solver's write target
CREATE TABLE assignment (
  id UUID PRIMARY KEY,
  timetable_version_id UUID REFERENCES timetable_version(id),
  staff_profile_id UUID REFERENCES staff_profile(id),
  course_id UUID REFERENCES course(id),
  cohort_id UUID REFERENCES cohort(id),
  room_id UUID REFERENCES room(id),
  slot_start INT NOT NULL,          -- Encoded as (weekday * periods_per_day + period_index)
  slot_span INT NOT NULL DEFAULT 1 -- >1 for multi-period lab blocks
);
```

```
-- The rule engine's storage
CREATE TABLE constraint_rule (
  id UUID PRIMARY KEY,
  tenant_id UUID REFERENCES tenant(id),
```

```

rule_type TEXT NOT NULL, -- Closed enum (see Section 7.3)
scope TEXT NOT NULL,
target_id UUID,
threshold NUMERIC,
unit TEXT,
polarity TEXT,
weight NUMERIC DEFAULT NULL, -- NULL = hard constraint; non-null = soft
source TEXT CHECK (source IN ('structured', 'nl')),
raw_input_text TEXT,
status TEXT DEFAULT 'confirmed'
);

```

7. The Constraint Engine

This is the core innovation and the actual product. Everything else in the system is scaffolding around these constraint lists.

7.1 Hard Constraints (Zero Violations — Always Guaranteed)

Each constraint is enforced both inside the solver AND independently re-verified afterward:

ID	Constraint	CP-SAT Formulation	Implementation
H1	No faculty double-booked in same slot	$\forall f, s: \sum \text{assign}[f, c, k, r, s] \leq 1$	<code>build_h1_no_faculty_double_booking()</code>
H2	No cohort double-booked in same slot	$\forall k, s: \sum \text{assign}[f, c, k, r, s] \leq 1$	<code>build_h2_no_cohort_double_booking()</code>
H3	No room double-booked in same slot	$\forall r, s: \sum \text{assign}[f, c, k, r, s] \leq 1$	<code>build_h3_no_room_double_booking()</code>
H4	Faculty not scheduled during unavailable blocks	<code>assign[f,,,*,s] = 0 for blocked s</code>	<code>build_h4_unavailable_blocks()</code>

ID	Constraint	CP-SAT Formulation	Implementation
H5	Faculty only assigned to eligible (course, cohort) pairs	Variables only created for eligible combos	Enforced by construction
H6	Weekly hours per (course, cohort) exactly met	$\sum \text{assign} = \text{required_hours}[c,k]$	<code>build_h6_hours_match()</code>
H7	Slots within valid period range for day/shift	Variables only created for valid ranges	Enforced by construction
H8	Faculty weekly/daily workload cap never exceeded	$\sum \text{assign} \leq \text{cap}[f]$ per week and per day	<code>build_h8_workload_cap()</code>
H9	Room type/equipment matches course requirement	Variables only created for matching pairs	Enforced by construction
H10	Shared lab resource capacity not exceeded	Lab rooms at slot s: $\sum \text{assign} \leq \text{room_count}$	<code>build_h10_shared_lab()</code>
H11	No student double-booking across elective combination	Cross-referenced via enrollment records	<code>build_h11_student_overlap()</code>
H12	(Exam) No student has overlapping exam sessions	Same as H11, applied to exams	<code>build_h12_exam_overlap()</code>
H13	(Exam) Room seating capacity not exceeded	$\text{enrollment_count} \leq \text{room.capacity}$	<code>build_h13_exam_capacity()</code>
H14	(Exam) Invigilator not	Same as H1/H8, applied to	<code>build_h14_invigilator()</code>

ID	Constraint	CP-SAT Formulation	Implementation
	double-booked; within cap	invigilators	
H15	Block courses: N consecutive periods, same room/day	Occupancy $s..s+n-1$ in all sums; no day-boundary crossing	<code>build_h15_block_size()</code>

7.2 Soft Constraints (Weighted Objective — Optimized)

These are penalty terms added to the solver's objective function. The solver minimizes total penalty while staying within the hard-constraint-feasible space:

ID	Constraint	Optimization Target
S1	Minimize idle gaps for faculty and students	Gap penalty per person per day
S2	Balance faculty load evenly across the week	Variance in daily teaching load
S3	Distribute course hours across different days	Penalty for same-day stacking
S4	Prefer core/high-focus subjects earlier in the day	Slot-position weighting
S5	Minimize same-course-twice-in-one-day	Double-scheduling penalty
S6	Prefer parallel batch sessions aligned to same slot	Alignment bonus
S7	Maximize room-utilization efficiency	Unused-capacity penalty
S8	Respect faculty time-of-day preferences	Preference mismatch penalty
S9	(Exam) Spread student exams evenly; minimize consecutive-day exams	Gap-day bonus

7.3 Constraint Rule Type Enumeration (Closed Set)

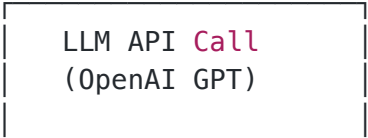
The NL parser and structured form can ONLY produce values from this closed enum. Anything unmapable routes to `unsupported` → manual structured form:

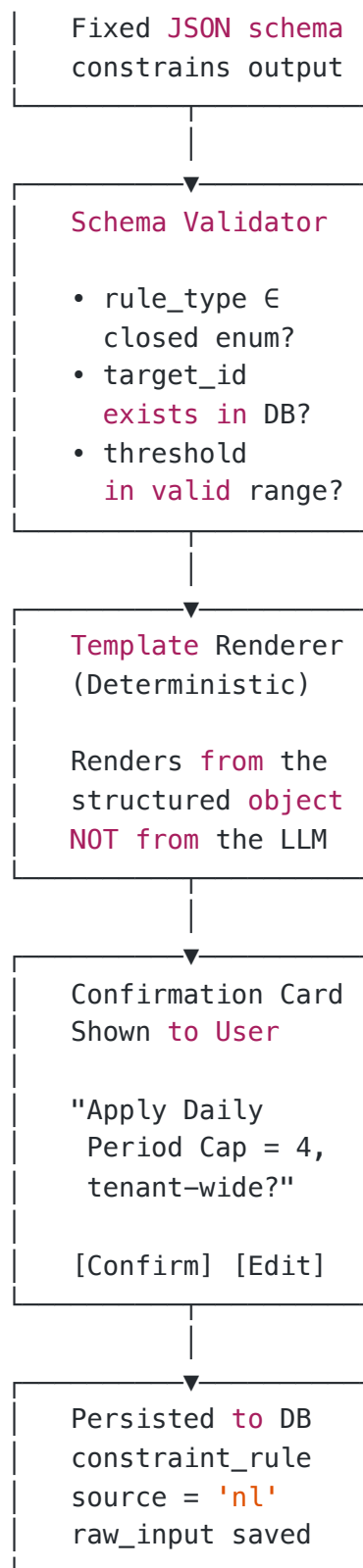
rule_type	unit	Maps to	Example phrasing
max_periods_per_day	periods	H8	"No faculty teaches more than 4 periods a day"
max_periods_per_week	periods	H8	"Faculty should teach at most 20 periods a week"
min_gap_between_periods	periods	S1	"Give faculty at least one free period between teaching blocks"
no_consecutive_same_course	boolean	S5	"Don't put the same subject back-to-back"
preferred_time_of_day	morning/afternoon	S4, S8	"Science should be scheduled in the morning"
balance_load_across_week	boolean	S2	"Spread faculty load evenly across the week"
room_utilization_priority	weight	S7	"Prioritize using all rooms evenly"
elective_no_overlap_core	boolean	H11	"Electives should never clash with core classes"
exam_min_gap_days	days	S9	"Give students at least one day between exams"
unsupported	—	none	Fallback → manual structured form

8. Natural Language Rule Builder

8.1 Pipeline Architecture

User types: "No faculty teaches more than 4 periods a day"





8.2 Critical Design Principle

The confirmation text is template-rendered from the structured JSON object — it is **NEVER** re-generated by the LLM. This guarantees that what the user sees and confirms is exactly what gets stored and fed to the solver. There is no hallucination risk in the confirmation step.

8.3 Unsupported Rule Handling

If the LLM cannot map the user's input to a known `rule_type` from the closed enum, the system:

- 1. Returns `rule_type: "unsupported"`
- 2. Shows the user a manual structured form instead
- 3. Never silently drops or invents a new constraint type

9. The Solver Pipeline

9.1 The CP-SAT Decision Variable

The core of the solver is a single binary decision variable:

```
assign[f, c, k, r, s] ∈ {0, 1}
```

Symbol	Meaning
f	Staff profile (faculty member)
c	Course
k	Cohort
r	Room
s	Time slot (block starting position, encoded as weekday × periods_per_day + period_index)

Meaning: "Faculty f teaches course c to cohort k in room r, starting at slot s."

9.2 Variable Pruning (Correctness by Construction)

Variables are **only created** for combinations that pass these filters:

- (f, c, k) must exist in the `eligibility` table → **enforces H5 by construction**
- (c, r) must have matching room type → **enforces H9 by construction**
- s must be within the valid period range for that day/shift → **enforces H7 by construction**

This means infeasible assignments don't exist as variables — the solver never even considers them.

9.3 Solve Process

1. `build_solver_input()`
 - |— Load all faculty, courses, cohorts, rooms from DB
 - |— Load eligibility mappings
 - |— Load period templates
 - |— Load constraint_rules
 - |— Load staff_availability_blocks
2. Create CP-SAT Model
 - |— Create BoolVar for each valid (f,c,k,r,s) tuple
 - |— build_h1() through build_h15() – add hard constraints
 - |— build_soft_objective() – add S1–S9 as weighted penalties
 - |— If previous solution exists: AddHint() for warm start
3. `Solver.Solve(timeout=SOLVER_TIMEOUT_SECONDS)`
 - |— If OPTIMAL or FEASIBLE: extract assignments
 - |— If INFEASIBLE: return error with diagnostic info
 - |— If timeout: return best-effort with soft shortfalls surfaced
4. `conflict_checker.verify(assignments)`
 - |— Re-validate H1–H15 using SEPARATE code
 - |— Return violations list (should be empty)
 - |— If violations found: flag as system error, don't save
5. Save assignments to DB as DRAFT timetable_version

9.4 Incremental Regeneration

When a small change occurs (e.g., a faculty member becomes unavailable), the system uses CP-SAT's `AddHint()` to seed the re-solve with the previous solution's variable values. This means:

- The solver starts from a known-good point, not from scratch
- Small changes typically resolve in seconds, not minutes
- Hard constraints are still fully guaranteed in the new solution

10. Independent Conflict Verification

This is a non-negotiable architectural invariant: **solver output is never trusted on its own.**

10.1 Why a Separate Checker?

A bug in the solver's constraint construction would produce incorrect output that the solver itself would believe is correct. The independent conflict checker uses a completely different code path to validate the same rules:

What the Solver Does	What the Checker Does
Adds <code>AddAtMostOne</code> constraints to the CP-SAT model	Iterates over all assignments, builds per-slot dictionaries, checks for duplicates
Uses variable construction to enforce eligibility	Joins assignments against the eligibility table and flags mismatches
Encodes workload caps as linear constraints	Counts assignments per faculty per day/week and compares to caps

10.2 Checker Output

The conflict checker returns a structured report:

```
{
  "hard_violations": [],           // Must be empty for a valid timetable
  "soft_shortfalls": [             // Informational warnings
    {"type": "S1", "entity": "faculty_123", "detail": "2-period gap on Monday"}
  ],
  "verified_at": "2024-01-15T10:30:00Z"
}
```

If `hard_violations` is non-empty, the timetable version is flagged and cannot proceed to review.

11. Role-Based Access Control

11.1 Role Matrix

Action	Platform Super Admin	Institution Admin	Department Head	Reviewer	Faculty	Student
Onboard new tenant	✓	✗	✗	✗	✗	✗
CRUD master data	✓	✓	✓ (own dept)	✗	✗	✗
Create/edit constraint rules	✓	✓	✓ (own dept)	✗	✗	✗

Action	Platform Super Admin	Institution Admin	Department Head	Reviewer	Faculty	Student
Trigger generation	✓	✓	✓ (own dept)	✗	✗	✗
Manual edit assignments	✓	✓	✓ (own dept)	✗	✗	✗
Review & approve	✓	✗	✗	✓	✗	✗
Publish	✓	✓	✗	✗	✗	✗
View own schedule	✓	✓	✓	✓	✓	✓
Declare absence	✗	✗	✗	✗	✓	✗

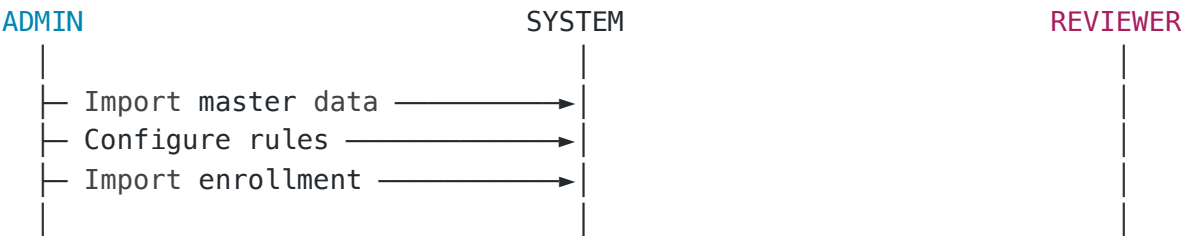
11.2 Enforcement

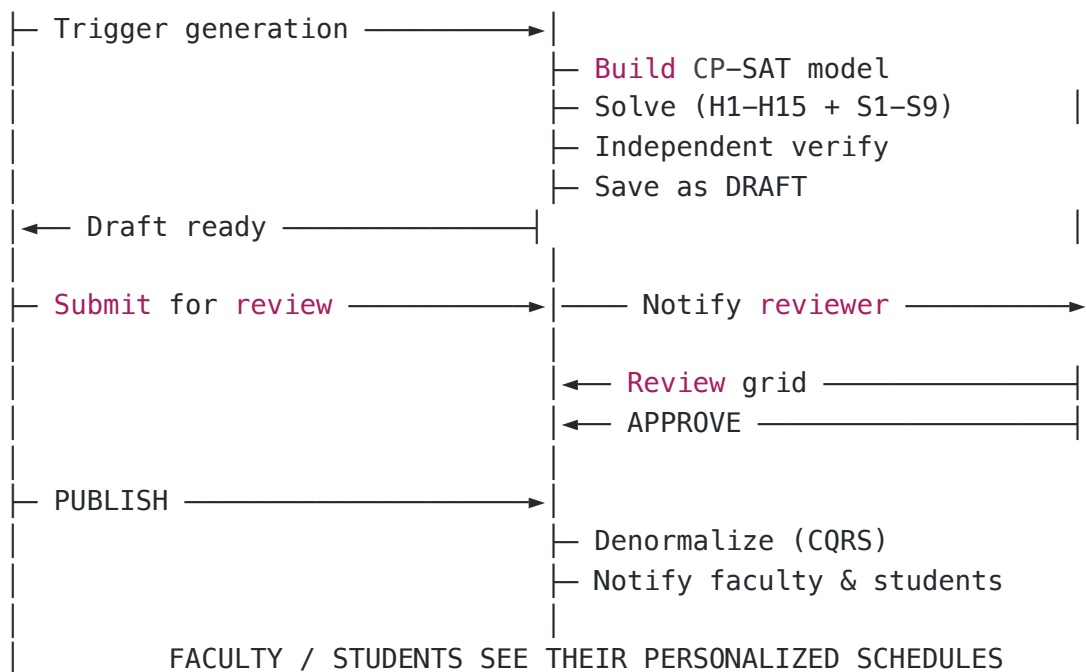
RBAC is enforced at the API layer via FastAPI dependency injection — not via frontend-only route guards. Every endpoint declares its required role(s) as a dependency:

```
@router.post("/generate")
async def generate_timetable(
    staff=Depends(require_role(["admin", "department_head"])),
    db=Depends(get_db),
):
    ...
```

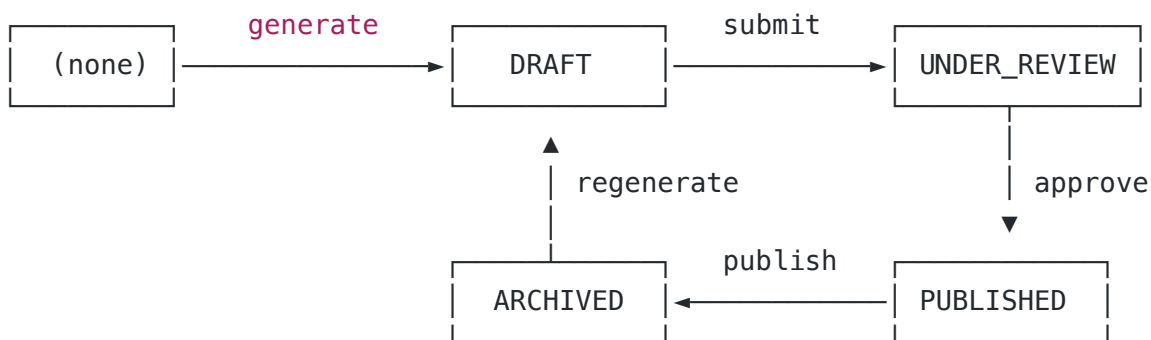
12. Core Workflows

12.1 The Once-Per-Term Cycle





12.2 The Publish State Machine



Non-negotiable invariant: No auto-publish, ever. Every tenant must pass through an explicit approve action before publish, regardless of institution size.

12.3 Substitution Workflow

Faculty reports absence → System finds eligible substitutes
 (same course eligibility + free **in** that slot + under workload cap)
 → Admin confirms substitute → Substitution logged → Affected users notified

13. Multi-Tenant Architecture & Security

13.1 Identity Model

One person (Identity) can hold staff/student profiles at multiple institutions:

```
Identity (alice@example.com)
├─ Staff Profile at Tenant A (Professor, Admin role)
└─ Staff Profile at Tenant B (Adjunct, Faculty role)
```

13.2 JWT Design

The JWT carries **only** `identity_id` (as sub claim) — **never** `tenant_id`. Tenant context is resolved per-request from the `{tenantId}` URL path parameter.

13.3 Database-Layer Isolation (RLS)

Every tenant-scoped request sets session variables before any query:

```
SET LOCAL app.tenant_id = '<tenantId>';
SET LOCAL app.identity_id = '<identity_id>';
```

Every table has an identical RLS policy:

```
CREATE POLICY tenant_isolation ON <table>
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

This means even if application code forgets a `WHERE tenant_id = ...`, the database itself will filter out other tenants' data.

14. API Design

14.1 Conventions

Convention	Detail	
Base path	<code>/api/v1/</code> — breaking changes ship as <code>/api/v2/</code> alongside	
Pagination	All list endpoints: <code>?cursor=&limit=</code> → <code>{items: [], next_cursor: string\`</code>	<code>null}</code>
Error envelope	<code>{error: {code, message, details}}</code> — consistent everywhere	

Convention	Detail	
Concurrency	version_no on timetable_version and assignment; stale writes return CONFLICT	

14.2 Core Endpoints

Method	Path	Purpose
POST	/tenants	Onboard new tenant
GET	/tenants/{id}/departments	List departments
GET	/tenants/{id}/courses	List courses
GET	/tenants/{id}/staff-profiles	List faculty
GET	/tenants/{id}/rooms	List rooms
POST	/tenants/{id}/rules	Create structured rule
POST	/tenants/{id}/rules/parse	Parse NL text → structured rule
POST	/tenants/{id}/rules/{ruleId}/confirm	Confirm parsed rule
POST	/tenants/{id}/timetables/generate	Trigger generation
GET	/tenants/{id}/timetables/{versionId}	Get timetable version
POST	/tenants/{id}/timetables/{versionId}/approve	Reviewer approval
POST	/tenants/{id}/timetables/{versionId}/publish	Publish (requires prior approval)
POST	/tenants/{id}/exams/generate	Trigger exam generation
GET	/tenants/{id}/reports/verification	Constraint verification report
GET	/me/tenants	List all tenants for current identity
POST	/tenants/{id}/import/faculty	CSV bulk import
POST	/tenants/{id}/import/rooms	CSV bulk import
POST	/tenants/{id}/import/courses	CSV bulk import

15. Frontend Design

15.1 Page Structure

Page	Role(s)	Purpose
Login	All	Email + password authentication
Dashboard	Admin, Dept Head	Overview with quick-action cards
Setup Wizard	Admin	Institution config, module toggles, CSV imports
Faculty Management	Admin, Dept Head	CRUD for faculty profiles with workload caps
Course Management	Admin, Dept Head	CRUD for courses with type, hours, block_size
Room Management	Admin, Dept Head	CRUD for rooms with capacity, equipment, accessibility
Rule Builder	Admin, Dept Head	Structured form + NL input with confirm card
Generate	Admin, Dept Head	Trigger solver, view progress
Review Timetable	Admin, Reviewer	Full grid view with conflict overlay
Publish	Admin	Confirm and publish after reviewer approval
Faculty View	Faculty	Personal schedule (read-only)
Student View	Student	Individualized schedule reflecting electives
Cohort View	Admin	Per-cohort schedule
Department View	Admin, Dept Head	Department-aggregated view
Reports	Admin	Room utilization, faculty load charts
Substitutions	Admin, Faculty	Absence logging, substitute suggestions
Exams Module	Admin	Exam generation, seating, invigilator roster

15.2 The Timetable Grid Component

The reusable `TimetableGrid` component renders a weekday × period matrix where each cell shows:

- Course name and code
- Assigned faculty
- Room number
- Visual indicators for labs (multi-period blocks), electives, and conflicts

16. Testing Strategy & Results

16.1 Solver Constraint Tests

Every hard constraint has a dedicated test file using a **violate-then-assert** pattern: construct a synthetic scenario designed to trigger a specific violation if the constraint weren't enforced, then assert the solver's output satisfies it:

Test File	Constraint Tested
<code>test_h1_no_faculty_double_booking.py</code>	H1: Faculty cannot be in two places at once
<code>test_h2_no_cohort_double_booking.py</code>	H2: Cohort cannot have two classes at once
<code>test_h3_no_room_double_booking.py</code>	H3: Room cannot host two classes at once
<code>test_h4_faculty_unavailable_blocks.py</code>	H4: Faculty not scheduled in blocked slots
<code>test_h5_faculty_eligibility.py</code>	H5: Only eligible faculty-course pairs
<code>test_h6_hours_match_required.py</code>	H6: Exact weekly hours met
<code>test_h7_valid_period_range.py</code>	H7: Slots within valid range
<code>test_h8_workload_cap.py</code>	H8: Workload caps respected
<code>test_h9_room_type_match.py</code>	H9: Room type matches course
<code>test_h10_shared_lab_capacity.py</code>	H10: Lab resource capacity
<code>test_h11_student_double_booking.py</code>	H11: No student elective overlap
<code>test_h12_student_exam_overlap.py</code>	H12: No exam overlap
<code>test_h13_exam_room_capacity.py</code>	H13: Exam seating capacity
<code>test_h14_invigilator_double_booking.py</code>	H14: Invigilator availability
<code>test_locked_assignments.py</code>	Locked assignments preserved during regen

Test File	Constraint Tested
test_rule_compiler.py	constraint_rule → solver parameters
test_soft_overrides.py	Custom soft constraint weights
test_exam_multi_room_split.py	Large exams split across multiple rooms

16.2 Running Tests

```
# Solver tests (18 files, all passing)
cd services/solver && pytest tests/ -v

# API integration tests
cd services/api && pytest tests/ -v
```

17. Performance Characteristics

Tier	Profile	Full Solve Target	Incremental Regen Target
Small (single school)	≤ 10 sections, ≤ 20 faculty	≤ 30 seconds	≤ 3 seconds
Medium (college, demo)	≥ 40 sections, ≥ 100 faculty	≤ 3 minutes	≤ 15 seconds
Large (university)	≥ 200 sections, ≥ 500 faculty	≤ 15 minutes	≤ 60 seconds

The demo seed data creates a medium-tier institution with 40+ cohorts and 100+ faculty profiles.

18. What Makes Schedulr Different

Aspect	Typical Scheduling Tool	Schedulr
Correctness guarantee	Best-effort heuristic	Mathematical proof via CP-SAT + independent re-verification

Aspect	Typical Scheduling Tool	Schedulr
Rule input	Checkboxes and dropdowns	Natural language with deterministic confirmation
Trust model	Trust the algorithm	Don't trust — independently re-prove every output
Publish control	Auto-save	Mandatory Draft → Review → Approve → Publish
Confirmation integrity	LLM generates confirmation text	Template-rendered from structured data (no hallucination risk)
Tenant isolation	Application-layer WHERE clauses	Database-layer RLS policies
Incremental updates	Full re-generate from scratch	Warm-started via AddHint()
Elective handling	Cohort-level only	Individualized per-student schedule
Extensibility	Monolithic	Solver is independently deployable; exam module slots in without changing solver core

19. Future Roadmap

Near-Term

- Drag-and-drop manual editing with live conflict overlay
- PDF/Excel/ICS export for all timetable views
- In-app notification inbox with configurable channels
- Enhanced analytics dashboards

Medium-Term

- Elective preference collection from students
- Enrollment demand forecasting from historical data
- What-if simulation UI (compare scenarios without committing)
- Schema-per-tenant isolation for large institutions

Long-Term

- Hostel & mess scheduling (reusing the same constraint engine)
- Native mobile apps (iOS/Android) with push notifications
- Multi-language UI (i18n)
- AI-powered proactive constraint suggestions
- Kubernetes production deployment with tenant-aware worker pools

This document is generated from the project source code and specification. For the complete 800-line technical specification, see `docs/PROJECT_SPEC.md` in the repository.